

MICROPROCESSOR AND COMPUTER ARCHITECTURE

Complete Assignment Solutions

Covers Units 1-9 and all 8085 laboratory programs

Student Name	Samir Puri
Program	BCA
Subject	Microprocessor and Computer Architecture
Document	Assignment Solution

Prepared as a study and submission-ready reference

Contents

1. Unit One - Introduction to Microprocessor
2. Unit Two - 8085 Microprocessor
3. 8085 Laboratory Programs
4. Unit Three - 8086 Microprocessor
5. Unit Four - Basic Computer Architecture and Design
6. Unit Five - Microprogrammed Control Unit
7. Unit Six - Central Processing Unit
8. Unit Seven - Computer Arithmetic
9. Unit Eight - Input/Output Organization and Memory Organization
10. Unit Nine - Pipelining

Note: The uploaded sheet contains a few duplicate/misnumbered questions. Every distinct question has been included; repeated questions are cross-referenced where appropriate.

Unit One: Introduction to Microprocessor

1. Define Microprocessor. Differentiate between Microprocessor and Microcontroller with example.

A microprocessor is a programmable, clock-driven electronic device that accepts binary data as input, processes it according to instructions stored in memory, and produces output. It contains the arithmetic and logic unit (ALU), control unit, and registers on a single integrated circuit. A microprocessor normally requires external memory and I/O devices to form a complete system.

A microcontroller is a complete small computer on a chip. In addition to a CPU, it contains program memory, data memory, timers, I/O ports, and often serial interfaces and ADC modules.

Basis	Microprocessor	Microcontroller
Integration	Mainly CPU on chip	CPU + memory + I/O peripherals on chip
External components	Needs more external RAM/ROM/I/O	Needs fewer external components
Typical use	General-purpose computing	Dedicated/embedded control
Cost/power	Usually higher system cost and power	Usually lower cost and power
Example	Intel 8085, 8086	8051, PIC16F877A, ATmega328P

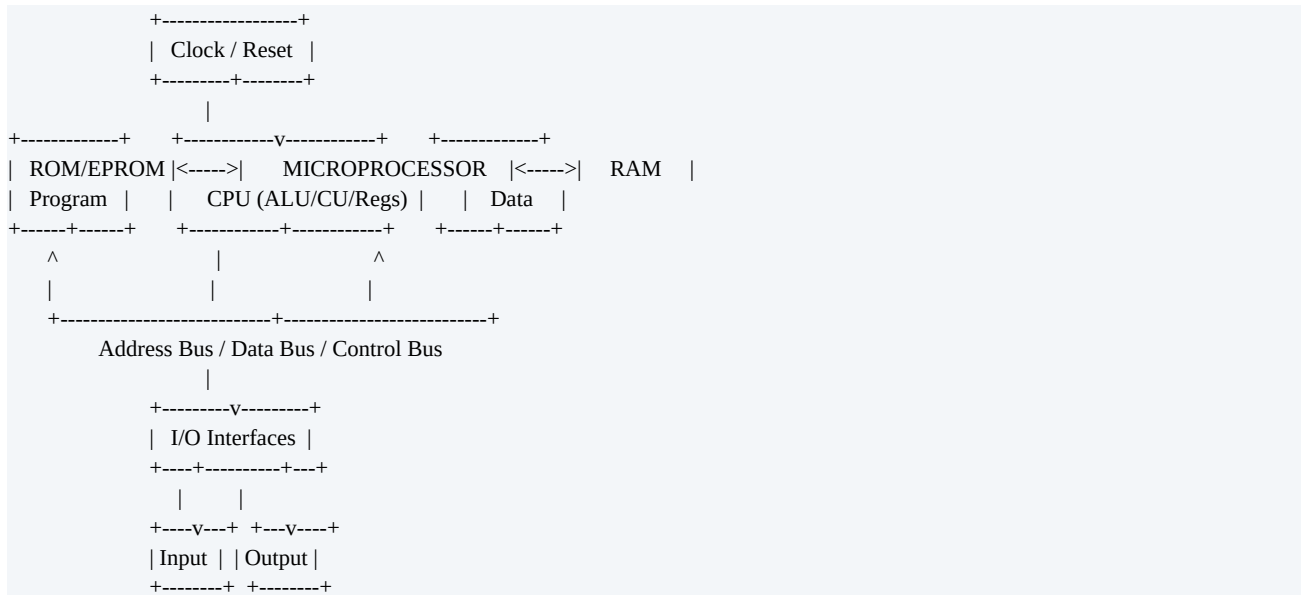
2. Explain microprocessor as a CPU.

The microprocessor acts as the Central Processing Unit (CPU) of a microcomputer system. It fetches instructions from memory, decodes them, executes the required operation, and controls data transfer among memory and I/O devices. The ALU performs arithmetic and logic operations, the register section holds temporary operands and results, and the control unit generates timing and control signals.

The CPU communicates with the rest of the system through the address bus, data bus, and control bus. During execution it repeatedly performs the fetch-decode-execute cycle. Therefore, the microprocessor is called the CPU because it is the main processing and coordinating element of the system.

3. Explain the organization of microprocessor based system with block diagram.

A microprocessor-based system is built around the CPU and includes memory, I/O devices, clock/reset circuitry, and buses. Program instructions are kept in ROM/EPROM or other non-volatile memory, while RAM stores temporary data and stack information. Input and output interfaces connect peripherals such as keyboards, displays, sensors, printers, and communication devices. Address decoding selects the required memory or I/O device.



Block diagram of a typical microprocessor-based system

4. Write the difference between microprocessor and microcontroller.

The essential difference is the level of integration. A microprocessor primarily provides the CPU and depends on external memory and peripheral chips, while a microcontroller integrates CPU, memory and I/O resources for embedded control. This makes microcontrollers compact, low-power and suitable for dedicated tasks, whereas microprocessors are preferred where greater flexibility, memory expansion, and general-purpose performance are needed.

Point	Microprocessor	Microcontroller
Chip contents	CPU mainly	CPU, RAM, ROM/Flash, timers, I/O
System size	Larger	Compact
Memory expansion	Usually easy and large	Usually limited/on-chip
Speed emphasis	High computation capability	Control-oriented
Applications	PCs, complex systems, trainers	Appliances, robots, vehicles, IoT
Examples	8085, 8086	8051, AVR, PIC

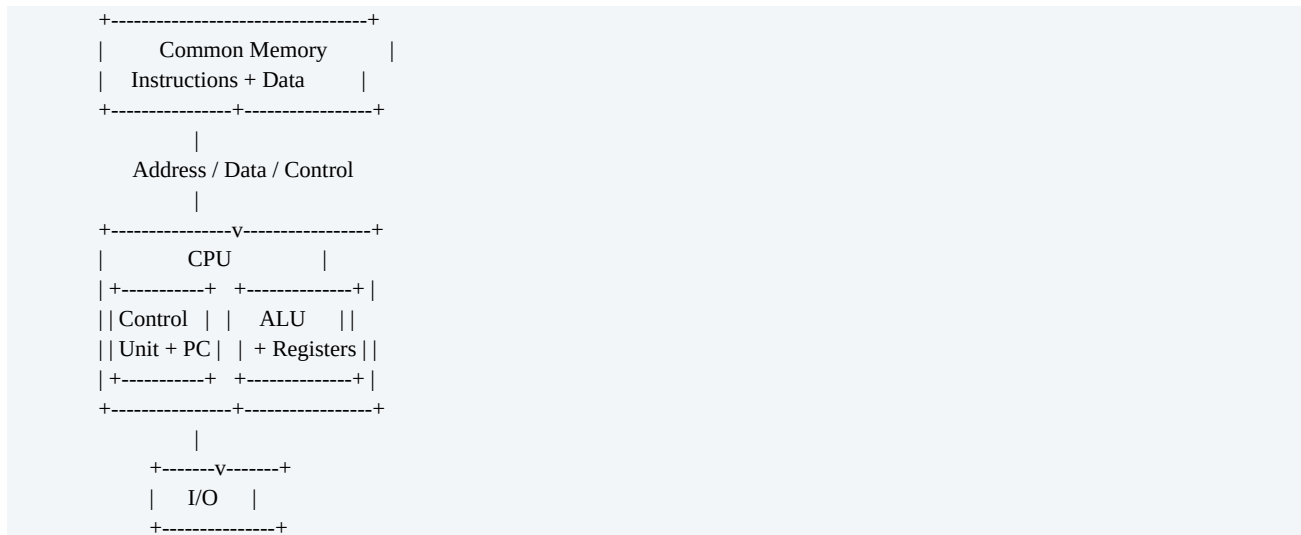
5. Write the function of data bus, address bus and control bus.

- Data bus: Carries actual data and instruction bytes between CPU, memory, and I/O devices. It is generally bidirectional. In 8085 it is 8 bits wide.
- Address bus: Carries the address of the memory location or I/O port selected by the CPU. It is normally unidirectional from CPU to external devices. In 8085 it is 16 bits wide, allowing 64 KB addressing.
- Control bus: Carries timing and control signals such as RD, WR, IO/M, interrupt acknowledgement, reset, and bus request/acknowledge. These signals specify what operation is taking place and synchronize the system.

6. Draw and explain the Von Neumann Architecture of microprocessor.

Von Neumann architecture uses one common memory to store both instructions and data. The CPU is connected to the common memory and I/O system through a shared set of buses. The program counter supplies the next instruction address, the control unit fetches and decodes the instruction, and the ALU executes it using registers and data from memory.

Because instructions and data share the same memory path, only one main memory transfer can normally occur at a time. This limitation is known as the Von Neumann bottleneck.



Von Neumann architecture

7. Differentiate Von Neumann Architecture with Harvard Architecture.

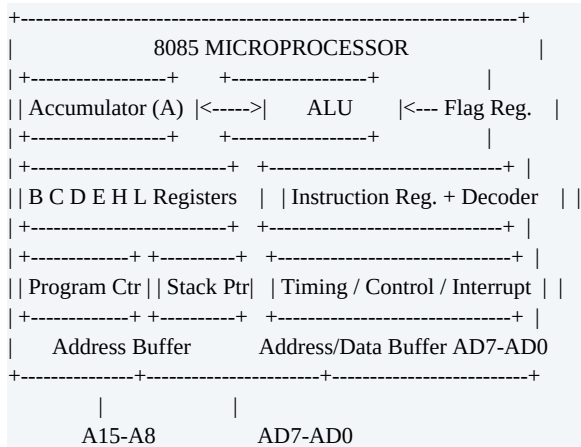
Feature	Von Neumann	Harvard
Memory	One memory for code and data	Separate instruction and data memories
Buses	Shared instruction/data bus	Separate instruction and data buses
Simultaneous access	Usually not possible	Instruction and data can be accessed simultaneously
Speed	Limited by shared path	Potentially faster
Complexity	Simpler and cheaper	More hardware complexity
Memory width	Usually same for code/data	Can use different widths
Example use	General-purpose computers	DSPs and many microcontrollers

Unit Two: 8085 Microprocessor

1. Explain the 8085 microprocessor architecture and its operations.

The Intel 8085 is an 8-bit microprocessor with an 8-bit data bus and 16-bit address bus. It can address 64 KB of memory. Its major blocks are the arithmetic and logic unit, accumulator, flag register, general-purpose registers B, C, D, E, H and L, program counter, stack pointer, instruction register/decoder, timing and control unit, interrupt control, serial I/O, and address/data buffers.

Operation begins when the program counter places an address on the bus. The processor fetches the opcode, decodes it, fetches any additional operand bytes, executes the operation in the ALU or data-transfer circuitry, updates flags if needed, and then proceeds to the next instruction. Machine cycles are synchronized by the clock and control signals.

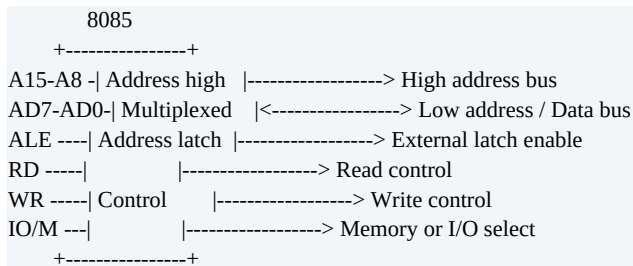


Simplified 8085 architecture

2. Explain the bus architecture of 8085 microprocessor.

The 8085 communicates with memory and I/O through three logical buses. The address bus is 16 bits. Lines A15-A8 carry the high-order address throughout the machine cycle, while AD7-AD0 are multiplexed: they carry A7-A0 during T1 and data D7-D0 during later T states. The ALE signal is used to latch the low-order address externally.

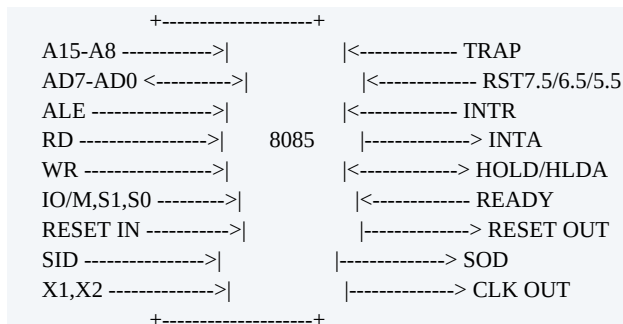
The 8-bit data bus AD7-AD0 is bidirectional and transfers opcodes, operands and results. The control/status bus includes RD, WR, IO/M, S1, S0, ALE, READY, HOLD and HLDA. Together, these signals identify and synchronize memory read/write, I/O read/write, interrupt and DMA operations.



8085 external bus organization

3. Explain the 8085 microprocessor signals with block diagram.

8085 pins can be grouped by function. Address/data signals are A15-A8 and multiplexed AD7-AD0. Control/status signals include ALE, RD, WR, IO/M, S0 and S1. Interrupt inputs are TRAP, RST 7.5, RST 6.5, RST 5.5 and INTR; INTA is the interrupt acknowledge output. DMA and wait-state signals are HOLD, HLDA and READY. Reset signals are RESET IN and RESET OUT. Serial I/O uses SID and SOD. Clock pins are X1, X2 and CLK OUT, and power pins are VCC and VSS.

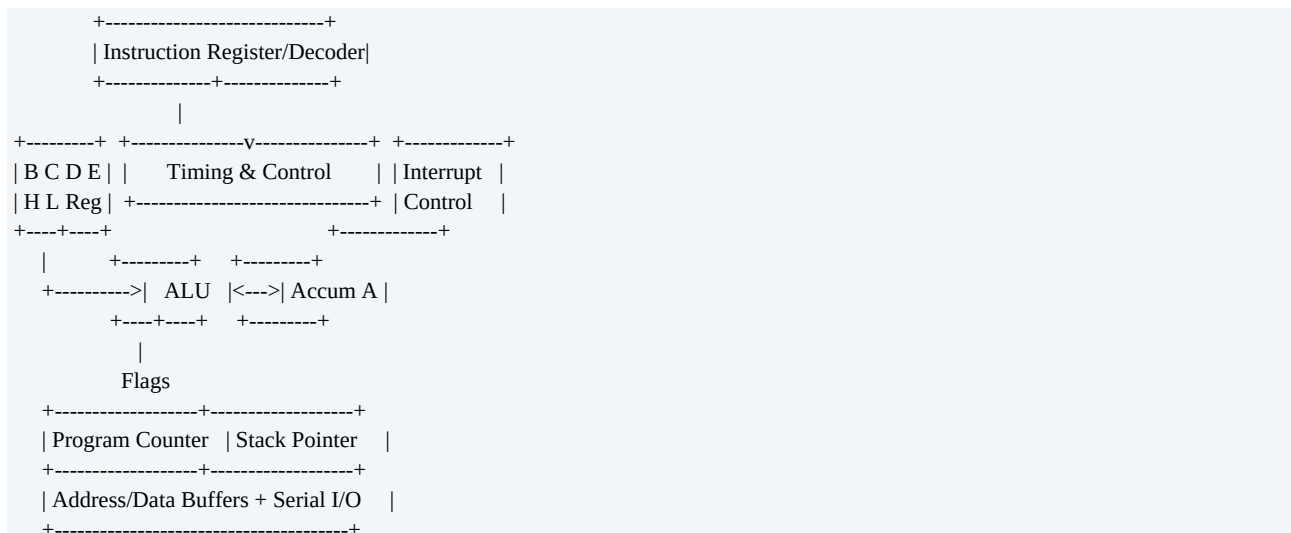


Functional grouping of important 8085 signals

4. Explain the 8085 microprocessor with its functional diagram.

Functionally, the 8085 contains a data-processing section and a control/bus section. The accumulator and ALU carry out 8-bit arithmetic and logical operations. The six general-purpose registers can be used separately or as BC, DE and HL register pairs. The program counter holds the next instruction address, while the stack pointer points to the current top of stack. The instruction decoder produces internal control signals from the fetched opcode.

The timing and control unit coordinates machine cycles and external read/write operations. Interrupt control gives prioritized response to external requests, and serial I/O provides one-bit input and output. Address and data buffers drive the external buses.



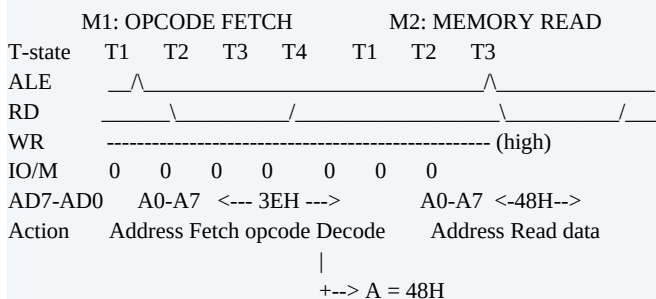
8085 functional block diagram

5. Explain the opcode fetch and memory read machine cycles for MVI A, 48H with timing for execution diagram.

MVI A,48H is a two-byte instruction. The first byte is the opcode 3EH and the second byte is immediate data 48H. Its execution requires two machine cycles: M1 opcode fetch and M2 memory read. The opcode fetch normally takes four T-states; the memory read takes three T-states, for a total of seven T-states.

During T1 of M1 the program counter address is placed on A15-A8 and AD7-AD0 and ALE goes high. In T2, RD goes low and memory places 3EH on the data bus. The opcode is decoded during the following state(s). The PC increments. M2 repeats address output for the next location; RD becomes low and 48H is read, then loaded into accumulator A.

MVI A,48H Byte 1 = 3EH (opcode), Byte 2 = 48H (data)



Timing sequence for MVI A,48H

6. Explain the 8085 microprocessor addressing modes with example.

An addressing mode specifies how an instruction locates its operand.

Mode	Meaning	Example
Immediate	Data is part of instruction	MVI A, 32H; ADI 05H
Register	Operand is in a CPU register	MOV A,B; ADD C
Direct	16-bit memory address is explicitly given	LDA 2500H; STA 3000H
Register indirect	Memory address is held in a register pair, usually HL	MOV A,M; MOV M,C
Implied/Inherent	Operand is implied by opcode	CMA; RAL; STC

7. Define Assembling. Explain the merits and demerits of Assembly Language Programming.

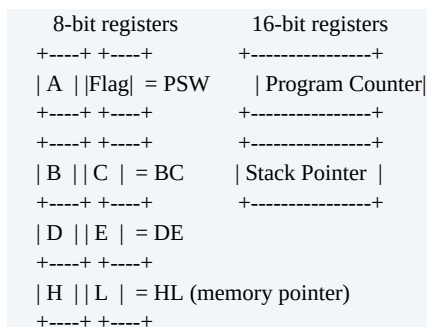
Assembling is the process of translating an assembly-language source program containing mnemonics, labels and symbolic operands into machine code. The software that performs this translation is called an assembler.

- Merits: efficient use of processor resources; compact and fast code; direct access to registers, flags, ports and special instructions; useful for time-critical and hardware-control routines; easier to debug than raw machine code because mnemonics and labels are used.
- Demerits: machine-dependent and non-portable; longer development time; programmer must understand hardware details; difficult to maintain for large applications; fewer high-level abstractions and libraries; errors involving registers, stack and memory are easy to make.

8. Explain the 8085 Programming model.

The programming model is the set of registers and flags visible to an assembly-language programmer. 8085 has accumulator A; six 8-bit registers B, C, D, E, H and L; a flag register; 16-bit program counter (PC); and 16-bit stack pointer (SP). BC, DE and HL can act as 16-bit register pairs. HL is especially important because it can work as a memory pointer, represented by M in instructions.

The flag register contains Sign (S), Zero (Z), Auxiliary Carry (AC), Parity (P), and Carry (CY). The accumulator and flag register together form the Processor Status Word (PSW).



Flag bits: S Z - AC - P - CY

8085 programming model

9. Classify the 8085 Instruction Set with example.

The 8085 instruction set is commonly classified according to operation performed.

Class	Purpose	Examples
Data transfer	Move data without changing its value	MOV, MVI, LXI, LDA, STA, LHLD, SHLD
Arithmetic	Addition, subtraction, increment, decrement	ADD, ADC, SUB, SBB, INR, DCR, DAD
Logical	AND, OR, XOR, compare, rotate,	ANA, ORA, XRA, CMP, RLC,

Class	Purpose	Examples
	complement	CMA
Branching	Change sequence of execution	JMP, JZ, JNZ, CALL, RET, RST
Machine control	Control processor state	NOP, HLT, EI, DI, SIM, RIM

10. Write short notes on: Instruction word size and Data format.

Instruction word size: 8085 instructions occupy one, two, or three bytes. A one-byte instruction contains the complete opcode and register information (for example MOV B,C). A two-byte instruction contains an opcode followed by 8-bit data or an 8-bit port address (for example MVI A,32H or IN 80H). A three-byte instruction contains an opcode followed by a 16-bit address or 16-bit immediate data, stored low-order byte first (for example LDA 2500H or LXI H,2050H).

Data format: 8085 processes data in 8-bit bytes. A byte may represent unsigned binary 00H-FFH, signed two's-complement values, BCD digits, characters, or bit patterns. Sixteen-bit data is normally handled as two consecutive bytes or in register pairs such as BC, DE, and HL. Multi-byte numeric values in 8085 memory are typically stored with the low-order byte at the lower address.

8085 Laboratory Programs

Assumptions: Standard Intel 8085 mnemonics are used. Example data, memory addresses, and port address 01H may be changed according to the trainer kit. HLT is used to terminate each standalone example.

Lab 1(a). Load an 8-bit data in a register.

```
MVI B, 5AH ; B <- 5AH
HLT
```

Result/Note: Register B contains 5AH.

Lab 1(b). Transfer data between registers.

```
MVI B, 32H
MOV A, B ; A <- B
HLT
```

Result/Note: The value in B is copied to A.

Lab 1(c). Transfer a specific data byte to a register.

```
MVI C, 45H ; C <- immediate byte 45H
HLT
```

Result/Note: Register C contains 45H.

Lab 1(d). Transfer a specific data byte to a memory location.

```
LXI H, 2500H ; HL points to 2500H
MVI M, 5AH ; [2500H] <- 5AH
HLT
```

Result/Note: Memory location 2500H contains 5AH.

Lab 1(e). Transfer data from a memory location to a register.

```
LXI H, 2500H
MOV A, M ; A <- [2500H]
HLT
```

Result/Note: Accumulator receives the byte stored at 2500H.

Lab 2. Add two hexadecimal numbers and display the sum at output PORT1.

```
MVI A, 25H
MVI B, 35H
ADD B      ; A = 25H + 35H = 5AH
OUT 01H    ; send sum to PORT1 (01H)
HLT
```

Result/Note: For the sample data, output is 5AH. Carry, if any, is indicated by CY flag.

Lab 3. Subtract two 8-bit hexadecimal numbers.

```
MVI A, 5AH ; minuend
MVI B, 25H ; subtrahend
SUB B      ; A = 35H
HLT
```

Result/Note: Accumulator contains the 8-bit difference. CY indicates borrow in 8085 subtraction.

Lab 4. Add two 16-bit hexadecimal numbers.

```
LXI H, 1234H ; first 16-bit number
LXI D, 1111H ; second 16-bit number
DAD D       ; HL = HL + DE
HLT
```

Result/Note: For the sample values, HL = 2345H. CY holds carry beyond 16 bits.

Lab 5. Subtract two 16-bit hexadecimal numbers.

```
LXI H, 3456H ; HL = minuend
LXI D, 1234H ; DE = subtrahend
MOV A, L
SUB E
MOV L, A
MOV A, H
SBB D ; subtract high byte with borrow
MOV H, A
HLT
```

Result/Note: For the sample values, HL = 2222H.

Lab 6. Multiply two 8-bit hexadecimal numbers.

```
MVI B, 06H ; multiplicand
MVI C, 05H ; multiplier
MVI A, 00H ; low byte of product
MVI H, 00H ; high byte of product
LOOP: ADD B
      JNC NEXT
      INR H
NEXT: DCR C
      JNZ LOOP
      MOV L, A
      HLT
```

Result/Note: Product is returned in HL. Example: 06H x 05H = 001EH.

Lab 7. Show division using 8-bit hexadecimal numbers.

```
MVI A, 24H ; dividend
MVI B, 06H ; divisor
MVI C, 00H ; quotient
LOOP: CMP B
      JC DONE
      SUB B
      INR C
      JMP LOOP
DONE: MOV D, A ; D = remainder
      MOV A, C ; A = quotient
      HLT
```

Result/Note: For 24H / 06H, quotient A = 06H and remainder D = 00H.

Lab 8. Check whether a specific data byte is even or odd.

```
MVI A, 37H
ANI 01H ; test least-significant bit
JZ EVEN
ODD: MVI B, 01H ; 01H means odd
      JMP STOP
EVEN: MVI B, 00H ; 00H means even
      STOP: HLT
```

Result/Note: The least-significant bit determines parity of an integer: 0 = even, 1 = odd.

Lab 9. Find the largest number in an array of data.

```
LXI H, 2500H ; 2500H stores count N
MOV C, M
INX H ; first data item
MOV A, M ; current largest
DCR C
JZ DONE
LOOP: INX H
      CMP M
      JNC SKIP ; if A >= M, keep A
      MOV A, M
SKIP: DCR C
      JNZ LOOP
DONE: STA 2600H ; save largest
      HLT
```

Result/Note: Array format: count at 2500H, data from 2501H onward; largest is stored at 2600H.

Lab 10. Find the smallest number in an array of data.

```
LXI H, 2500H ; 2500H stores count N
MOV C, M
INX H
MOV A, M ; current smallest
DCR C
JZ DONE
LOOP: INX H
      CMP M
      JC SKIP ; A < M
      JZ SKIP ; A = M
      MOV A, M ; A > M, update smallest
SKIP: DCR C
      JNZ LOOP
DONE: STA 2600H
      HLT
```

Result/Note: Smallest element is stored at 2600H.

Lab 11. Print/store Fibonacci series up to the 7th term.

```
LXI H, 2500H ; destination array
MVI A, 00H ; first term
MVI B, 01H ; second term
MVI C, 07H ; seven terms
LOOP: MOV M, A ; store current term
      INX H
      MOV D, A
      MOV A, B
      ADD D ; next = old A + old B
      MOV D, A
      MOV A, B ; shift sequence
      MOV B, D
      DCR C
      JNZ LOOP
      HLT
```

Result/Note: Memory 2500H onward receives 00H, 01H, 01H, 02H, 03H, 05H, 08H.

Lab 12. Find the sum of 16-bit numbers.

The wording in the sheet is ambiguous. A common lab interpretation is to add two 16-bit numbers stored in memory and save the 16-bit result. The following program does that.

```
LHLD 2500H ; first number: L=[2500], H=[2501]
XCHG ; first number -> DE
LHLD 2502H ; second number -> HL
DAD D ; HL = HL + DE
SHLD 2504H ; store result at 2504H-2505H
HLT
```

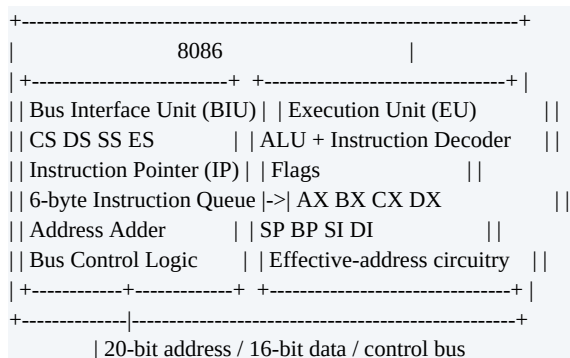
Result/Note: CY indicates overflow beyond 16 bits.

Unit Three: 8086 Microprocessor

1. Write and explain the logical block diagram of the 8086 microprocessor, describing BIU and EU and their sub-components.

The 8086 is organized into two major functional units: the Bus Interface Unit (BIU) and Execution Unit (EU). The BIU performs all external bus activities. It contains segment registers CS, DS, SS and ES, the Instruction Pointer (IP), address-generation circuitry, bus-control logic, and a six-byte instruction queue. It forms the 20-bit physical address as Segment x 10H + Offset.

The EU takes instruction bytes from the queue, decodes them, calculates effective offsets, and executes operations. It contains the ALU, instruction decoder/control circuit, flag register, general registers AX, BX, CX and DX, and pointer/index registers SP, BP, SI and DI. The separation permits overlapping instruction fetch and execution.



2. Explain the concept of pipelining in 8086.

8086 uses a simple two-stage pipeline created by the BIU/EU split. While the EU executes the current instruction, the BIU can fetch subsequent instruction bytes from memory whenever the external bus is free. These prefetched bytes are stored in a six-byte FIFO instruction queue. As a result, instruction fetch and execution overlap instead of occurring strictly one after another.

The queue is filled from sequential addresses. The EU removes bytes as it decodes instructions. If a branch, jump, call, return, or interrupt changes the flow of execution, the old prefetched bytes are no longer useful; the queue is flushed and fetching restarts from the new address. This overlap improves throughput but does not reduce every instruction's individual execution latency.

3. How does the instruction queue improve performance? What are the limitations of this approach?

The six-byte queue improves performance because memory fetch can take place in parallel with instruction execution. When the EU finishes an instruction, the next opcode is often already available in the queue, reducing idle bus/fetch time. This is especially effective for sequential code and longer EU operations during which the BIU has time to prefetch several bytes.

- Benefits: overlaps fetch and execute; improves instruction throughput; makes better use of otherwise idle bus cycles; reduces average waiting for sequential instruction bytes.
- Limitations: any control-transfer instruction can flush the queue; the queue is only six bytes; the BIU cannot prefetch while the EU needs the bus for memory operands; slow memory or frequent branches reduce the advantage; it is not a full multi-stage modern pipeline.

4. Compare 8085 and 8086 in terms of data bus width, address bus, memory capacity, instruction set, and architecture.

Feature	8085	8086
Data bus	8-bit	16-bit
Address bus	16-bit	20-bit
Maximum directly addressable memory	64 KB	1 MB
Architecture	Single processing section; no instruction queue	BIU + EU with 6-byte prefetch queue
Registers	Mostly 8-bit, some 16-bit pairs	Rich set of 16-bit registers
Memory organization	Linear 64 KB	Segmented memory
Instruction set	Smaller 8-bit instruction set	Larger, more powerful 16-bit instruction set
Pipelining	No	Simple fetch-execute overlap
Typical clock	About 3-6 MHz variants	5-10 MHz common original variants

Unit Four: Basic Computer Architecture and Design

1. What is Computer Architecture? Differentiate Computer Architecture with Computer Organization.

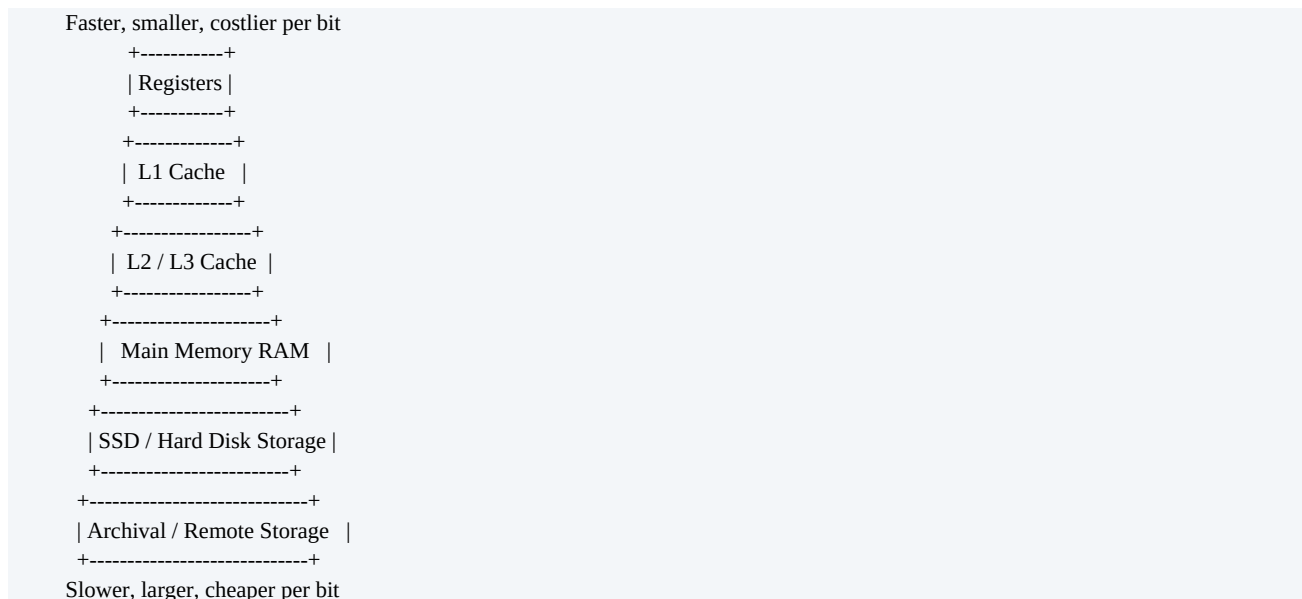
Computer architecture describes the attributes of a computer system that are visible to a programmer and that affect program execution. It includes the instruction set, data types, registers, addressing modes, I/O mechanisms,

and memory addressing model. Computer organization describes how the architectural specification is physically implemented using hardware units and their interconnections.

Basis	Computer Architecture	Computer Organization
Focus	What the computer does	How the computer is built to do it
Programmer visibility	Mostly visible	Mostly hidden
Examples	Instruction set, word size, addressing modes	Control signals, buses, cache technology, memory hardware
Design concern	Functional behavior and interface	Implementation and performance
Compatibility	Same architecture can run same machine code	Different organizations may implement same architecture

2. Explain the Memory Hierarchy with diagram.

Memory hierarchy arranges storage levels according to speed, cost per bit, and capacity. The fastest and most expensive storage is placed nearest the CPU, while larger and cheaper storage is farther away. Frequently used instructions and data are kept in higher levels according to the principles of temporal and spatial locality. This gives a system the effect of a large memory at a lower average access time.



Memory hierarchy

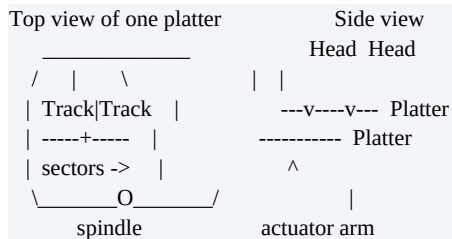
3. Explain the elements of cache design.

- Cache size: total storage capacity. A larger cache can reduce misses but may increase cost and access time.
- Block/line size: amount transferred between main memory and cache on a miss. Larger blocks exploit spatial locality but may waste space.
- Mapping function: determines where a memory block can be placed - direct mapped, fully associative, or set associative.
- Replacement policy: chooses which line to evict when a set is full, such as LRU, FIFO, or random.
- Write policy: write-through updates cache and memory immediately; write-back updates memory only when a dirty line is replaced.
- Number of cache levels: L1, L2 and L3 balance speed and capacity.
- Unified vs split cache: one cache can hold both data and instructions, or separate instruction and data caches can be used.
- Hit time, miss rate and miss penalty: the main performance measures used to evaluate a cache design.

4. Explain the internal structure of Hard Disk.

A hard disk drive stores data magnetically on rotating platters. Each platter has one or two recording surfaces divided into concentric tracks; tracks are divided into sectors. A spindle motor rotates the platters at a fixed speed. Read/write heads are mounted on an actuator arm and move radially to the required track. The actuator is controlled by a voice-coil mechanism. On-board electronics contain the disk controller, cache/buffer memory, servo logic and interface circuitry.

A cylinder is the set of tracks at the same radius on all platter surfaces. To access data, the drive performs seek time (move the head), rotational latency (wait for the desired sector), and data transfer.



Main parts: platters, spindle motor, read/write heads, actuator arm, voice-coil actuator, controller electronics, cache and host interface.

Simplified hard-disk structure

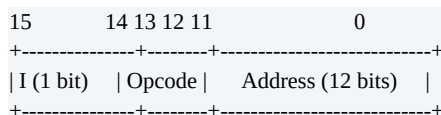
5. Explain the concept of Stored Program Organization.

Stored-program organization means that program instructions and data are represented in binary form and stored in memory. The CPU fetches an instruction from the address held in the program counter, decodes it, executes it, then normally advances the program counter to the next instruction. Because instructions reside in memory, a program can be changed simply by loading different instruction words rather than rewiring the computer.

This concept, associated with the classical Von Neumann model, enables general-purpose computing, loops, branches, subroutines, compilers and operating systems. The same memory system can therefore hold both the program and the data manipulated by that program.

6. Demonstrate the theory of direct and indirect address with instruction format diagrams.

In the basic computer instruction format, the most significant I bit indicates whether the 12-bit address field is used directly or indirectly for memory-reference instructions. With direct addressing (I=0), the address field itself is the effective address (EA). With indirect addressing (I=1), the address field points to a memory word that contains the effective address.



Direct addressing (I=0):

Instruction address field -----> Effective Address ----> Operand

Indirect addressing (I=1):

Instruction address field ----> Memory pointer ----> Effective Address ----> Operand

Direct and indirect addressing in a basic computer

7. Define registers. Explain the basic computer registers with their uses.

A register is a small, high-speed storage element inside the CPU used to hold data, addresses, instructions, or intermediate results. In the basic computer model, registers have specialized functions.

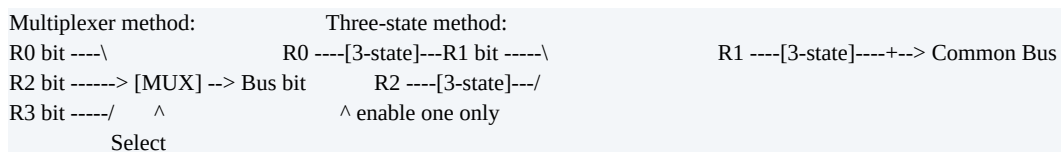
Register	Typical size	Use
AR - Address Register	12 bits	Holds memory address

Register	Typical size	Use
PC - Program Counter	12 bits	Address of next instruction
DR - Data Register	16 bits	Holds memory operand/data
AC - Accumulator	16 bits	Main ALU operand and result register
IR - Instruction Register	16 bits	Holds current instruction
TR - Temporary Register	16 bits	Temporary internal data
INPR - Input Register	8 bits	Receives input character/data
OUTR - Output Register	8 bits	Holds output character/data
SC - Sequence Counter	Usually 4 bits	Generates timing steps T0, T1, ...

8. Define Bus System. Explain bus system construction using Multiplexers and Three-state buffer gates.

A bus system is a shared set of lines used to transfer binary information among registers, memory and other functional units. Instead of providing a dedicated connection between every pair of registers, a common bus reduces wiring and is controlled so that only one source drives the bus at a time.

Using multiplexers, each bus bit is produced by a multiplexer that selects the corresponding bit from one of several registers. Common select lines choose the source register. Using three-state buffers, each register connects to the bus through buffers that can output 0, output 1, or enter a high-impedance state. Only the selected register's enable line is asserted; all others are electrically disconnected.



For an n-bit bus, the same selection is repeated for every bit position.

Common bus construction mechanisms

Additional question from the sheet: Explain the instruction set design issues.

Instruction-set design determines the interface between software and CPU hardware. Important issues include the operation repertoire (which arithmetic, logical, control and system operations are supported), number and type of operands, register organization, instruction length, encoding space, data types, addressing modes, memory addressing, and branch mechanisms. Designers also choose between fixed-length and variable-length instructions and decide how much work each instruction should perform.

Other trade-offs include code density versus simple decoding, orthogonality versus opcode-space cost, compatibility with compilers, support for procedure calls and interrupts, ease of pipelining, and backward compatibility. RISC designs emphasize simple regular operations and many registers, while CISC designs often provide richer instructions and addressing modes.

9. Explain the structure of control unit of basic computer.

The control unit coordinates all register transfers and micro-operations required to execute instructions. In a basic hardwired computer, an instruction register provides opcode bits to a decoder, a sequence counter is decoded to produce timing signals T0, T1, T2 and so on, and control logic combines decoded instruction bits, timing signals, flags and status conditions to generate register load/increment/clear signals, ALU functions, memory read/write controls and bus selections.

IR opcode ---> +-----+ ---> D0...D7 instruction decoder outputs

| Opcode Decoder |

+-----+

Clock ---> Sequence Counter ---> Timing Decoder ---> T0,T1,T2,...

Flags / I bit / status -----+

v

+-----+

| Control Logic |

+-----+

| | | |

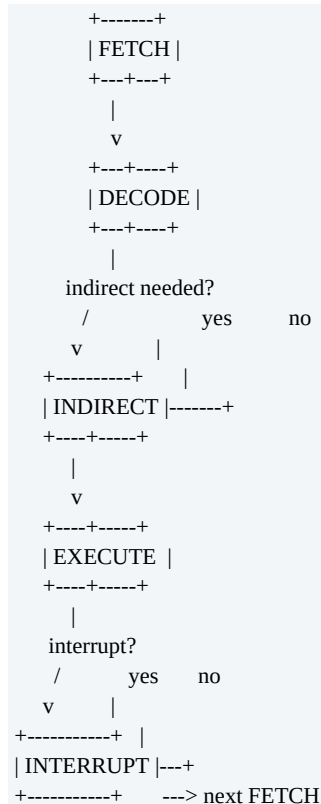
v v v v v

Registers / ALU / Memory / Bus

Basic hardwired control structure

10. Define Instruction Cycle. Explain the instruction cycle with state diagram.

The instruction cycle is the complete sequence of CPU operations required to fetch and execute one instruction. A general instruction cycle contains fetch, decode/effective-address determination, execute, and interrupt check/service. In a basic computer, an indirect cycle is inserted when a memory-reference instruction uses indirect addressing. After execution, the CPU checks whether an enabled interrupt request is pending; if so, it enters the interrupt cycle before fetching the next instruction.



General instruction-cycle state diagram

11. Define Micro-operation.

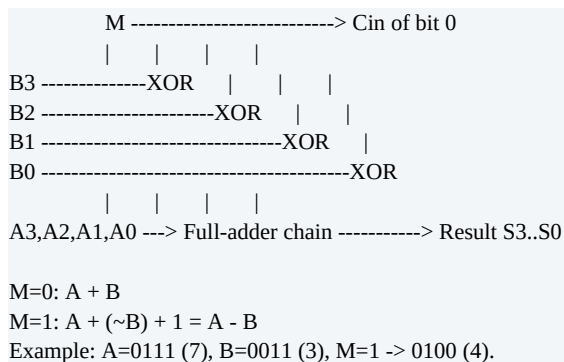
A micro-operation is an elementary operation performed on data stored in registers, usually completed within one clock pulse. Micro-operations are the low-level steps from which machine instructions are constructed. The main classes are register-transfer micro-operations (for example $R2 \leftarrow R1$), arithmetic micro-operations ($R3 \leftarrow R1 + R2$), logic micro-operations ($R1 \leftarrow R1 \text{ AND } R2$), and shift micro-operations ($R1 \leftarrow \text{shl } R1$). Control signals generated by the control unit select and sequence these operations.

12. Explain the difference between logical and arithmetic shift operations.

Point	Logical Shift	Arithmetic Shift
Purpose	Treats word mainly as unsigned/bit pattern	Preserves signed two's-complement meaning where possible
Right shift fill	Zero enters MSB	Original sign bit is replicated into MSB
Left shift fill	Zero enters LSB	Usually zero enters LSB; overflow may change sign
Use	Bit manipulation, unsigned multiply/divide by 2	Signed multiply/divide by powers of 2
Example 8-bit right	10110000 -> 01011000	10110000 -> 11011000

13. Draw and explain adder-subtractor with example.

A binary adder-subtractor can perform both operations using n full adders and XOR gates. A mode input M is connected to one XOR input for every B bit and also to the carry input of the least significant full adder. When $M=0$, B passes unchanged and $C_{in}=0$, so the circuit calculates $A+B$. When $M=1$, each B bit is complemented and $C_{in}=1$, so the circuit calculates $A + B' + 1 = A-B$ by two's complement.



4-bit adder-subtractor principle

14. Explain the hardware implementation of arithmetic, logical and shift microoperations with diagram.

A combined arithmetic-logic-shift unit places an arithmetic circuit and a logic circuit in parallel. Multiplexers select one function result, and a shifter optionally shifts the selected result before loading the destination register. Control inputs determine source registers, arithmetic function, logic function and shift mode. A common ALU can therefore realize operations such as add, subtract, increment, AND, OR, XOR, complement, logical shift and arithmetic shift within a register-transfer system.



Combined arithmetic, logic and shift hardware

15. Write the applications of logical microoperation.

- Selective set: OR a register with a mask to force selected bits to 1.
- Selective clear: AND with a mask containing 0s at positions to be cleared.

- Selective complement: XOR with a mask to toggle selected bits.
- Masking: retain only selected fields of a word using AND.
- Insertion: combine a cleared field with new bits using AND/OR operations.
- Comparison: XOR two bit patterns; a zero result indicates equality.
- Bit testing: AND a value with a one-bit mask to test a flag or condition.
- Assembly/disassembly of fields: pack or extract control, status, character, or device fields.

Unit Five: Microprogrammed Control Unit

1. Explain the design procedure of Accumulator Logic.

Accumulator logic is the hardware surrounding the AC register that implements the arithmetic, logic and shift micro-operations specified by the instruction set. A systematic design procedure is: identify all required AC micro-operations; express each as a register-transfer statement; group operations that can share hardware; design the arithmetic circuit, logic circuit and shifter; assign control variables to select each operation; connect the selected result to the AC input; and derive the Boolean control equations for AC load, increment and clear.

The final design is verified by checking that each instruction/timing condition activates exactly the required function without causing conflicting writes. In a basic computer, AC is also linked with the E (carry/extend) flip-flop for arithmetic and circular shift operations.

2. Explain the gate structure for controlling the LD, INR, and CLR of Accumulator.

A register such as AC commonly has three control inputs: LD (load), INR (increment), and CLR (clear). The control unit derives each signal as the OR of all instruction/timing conditions that require that operation. For example, several memory-reference or register-reference instructions may cause AC to load a new ALU result, while only particular micro-operations request clear or increment. AND gates detect each required condition and their outputs are ORed to form LD, INR, or CLR.

Priority must be defined if controls could be asserted together; in well-designed control logic, mutually exclusive timing equations prevent unintended simultaneous operations.

```
Condition 1 --AND--Condition 2 --AND---+---> OR ---> LD(AC)
Condition 3 --AND--/
```

```
Condition 4 --AND-----> OR ---> INR(AC)
Condition 5 --AND-----> OR ---> CLR(AC)
```

Instruction decoder outputs + timing signals + flags are the AND-gate inputs.

Control-gate structure for accumulator controls

3. Explain the components of ALU with their functions.

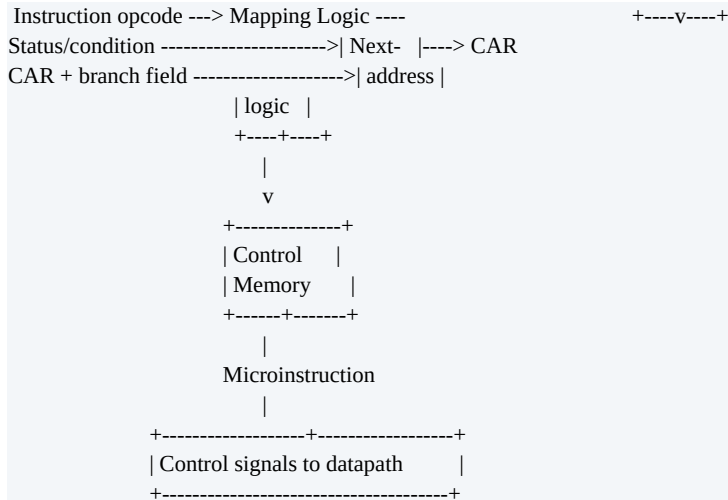
An Arithmetic Logic Unit is the combinational section of the CPU that transforms binary operands. Its main components are an arithmetic circuit, logic circuit, function-selection multiplexers and often a shifter. The arithmetic circuit uses adders to implement add, subtract, increment, decrement and related functions. The logic circuit implements bitwise AND, OR, XOR and complement. Multiplexers select the desired function according to control bits. A shifter performs left/right logical or arithmetic shifts. Status logic produces flags such as zero, sign, carry and overflow from the result.

4. Define Control Unit. Explain the organization of Microprogrammed Control Unit.

The control unit is the part of the CPU that directs the sequence of register transfers, ALU operations, memory accesses and I/O actions needed to execute instructions. In a microprogrammed control unit, control signals are represented as fields or bits in microinstructions stored in a control memory.

The main blocks are control memory, Control Address Register (CAR), microinstruction/control data register, a microprogram sequencer or next-address generator, mapping logic, condition/status inputs, and decoding/control-

signal logic. CAR selects a microinstruction. That microinstruction activates datapath control signals and also contains information used to choose the address of the next microinstruction.



Organization of a microprogrammed control unit

5. Differentiate between Hardwired and Microprogrammed Control Design.

Feature	Hardwired Control	Microprogrammed Control
Implementation	Gates, decoders, flip-flops, FSM logic	Microinstructions stored in control memory
Speed	Generally faster	Generally slower due to control-memory access
Modification	Difficult; hardware redesign may be required	Easier by changing microcode
Complex ISA	Becomes difficult as complexity increases	Well suited to complex instruction sets
Design/debug	Less flexible	More systematic and easier to debug
Typical association	RISC-oriented designs often favor hardwired control	Historically common in CISC designs

6. Explain the address sequencing procedure.

Address sequencing is the process of determining which microinstruction address will be loaded into the Control Address Register next. The next address may come from: CAR+1 for sequential execution, an address field in the current microinstruction for a branch, a mapped starting address derived from the machine instruction opcode, or a saved return address when a microprogram subroutine returns.

A microprogram sequencer selects among these sources according to branch-control bits and processor conditions. Thus each microinstruction both controls the datapath and helps determine the next step in the control routine.

7. Explain the conditional branching mechanism.

A conditional microprogram branch changes the next microinstruction address only when a selected status condition has a specified value. The microinstruction contains a condition-select field, a branch-control field and usually a target address. A multiplexer selects one status input, such as Zero, Carry, Sign, interrupt request, or an

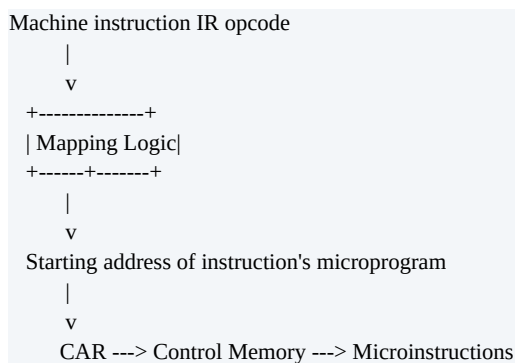
instruction bit. The sequencer tests the condition: if true, CAR receives the branch address; if false, CAR normally receives CAR+1.

This mechanism allows one microprogram to make decisions, implement loops, test ALU results, and select alternative control sequences without duplicating large portions of control memory.

8-9. What is mapping of instructions? Explain mapping from instruction code to microinstruction address.

Mapping is the conversion of a machine-instruction opcode into the starting address of the microprogram routine that implements that instruction. When the machine instruction is decoded, its opcode bits are passed to mapping logic. The mapper places or transforms those bits into a wider control-memory address, often by appending fixed low-order zeros or using a ROM/PLA lookup. The resulting address is loaded into CAR.

For example, if each machine instruction is assigned a block of four microinstructions, two low address bits can be forced to 00 and the opcode can form upper bits. After the routine begins, normal sequencing and conditional branching execute its individual micro-operations. At the end, control branches back to the common instruction-fetch microprogram.



Instruction-code to microprogram-address mapping

10. Define Microprogram. Differentiate Symbolic and Binary microprogram with example.

A microprogram is an ordered sequence of microinstructions stored in control memory that generates the control signals required to execute a machine instruction or common CPU routine. A symbolic microprogram is written using readable names for micro-operations, conditions and labels; a microassembler translates it into binary microinstructions stored in control memory.

Form	Description	Example
Symbolic	Uses labels and register-transfer notation	FETCH: AR <- PC, READ
Binary	Encoded control bits and next-address fields	0010 1011 0001 ...

11. Explain the basic requirements for designing a control unit.

- Complete specification of the instruction set and required instruction cycles.
- Datapath description: registers, buses, ALU functions, memory and I/O controls.
- A list of elementary micro-operations needed for each instruction.
- Correct sequencing and timing of those micro-operations.
- Condition and status handling for branches, flags, interrupts and exceptions.
- Generation of non-conflicting control signals for every clock step.
- Performance, cost, complexity, testability and reliability constraints.
- For microprogrammed control: control-word format, control-memory size, next-address mechanism and opcode mapping.

12. Explain the structure of Control Unit.

A control unit receives the current instruction, timing/clock information and status conditions, and produces the control signals that coordinate the datapath. Its conceptual structure includes the instruction register and decoder, sequencing mechanism, timing generator, condition logic and control-signal generator. In hardwired control, these are implemented directly as sequential/combinational logic. In microprogrammed control, the sequencing and control-signal patterns are stored in control memory and read as microinstructions.

Instruction Register ---> Decoder -----Clock ---> Sequencer / Timing -----+---> Control Signal Generator ---> Datapath
Flags / Interrupts / Conditions -----/

Hardwired: generator = logic network/FSM

Microprogrammed: generator = control memory + microinstruction decoder

Conceptual control-unit structure

13. Write the role of microprogram sequencer in microprogrammed control unit.

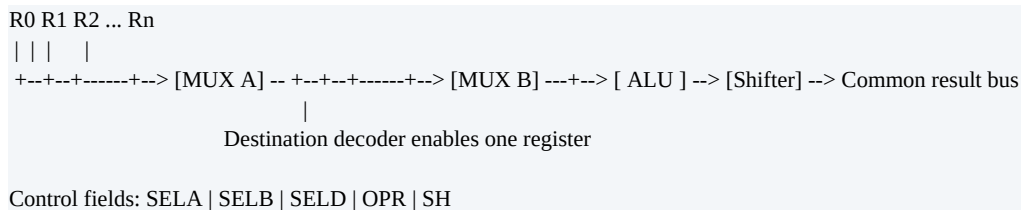
The microprogram sequencer determines the address of the next microinstruction. It supports sequential stepping, unconditional branches, conditional branches, opcode mapping, microprogram subroutine call/return and entry to common routines such as fetch or interrupt service. It evaluates branch fields in the current microinstruction together with CPU status conditions and selects the appropriate source for CAR. Therefore, the sequencer provides the flow-control mechanism of the microprogram, analogous to the program counter and branch logic used for ordinary machine instructions.

Unit Six: Central Processing Unit

1. Explain the register set with common ALU / general register organization.

A general register organization uses a set of CPU registers connected to a common ALU through shared buses. Two multiplexers select any two registers as source operands for the ALU. The ALU performs the function specified by control bits, an optional shifter processes the result, and a decoder enables one destination register to load the result. A control word therefore identifies source A, source B, destination register, ALU function and shift function.

This organization provides flexible register-to-register operations while avoiding a separate arithmetic unit for every register pair.



General register organization with common ALU

2. Define control word. Explain the procedure for determining control word for a specific operation.

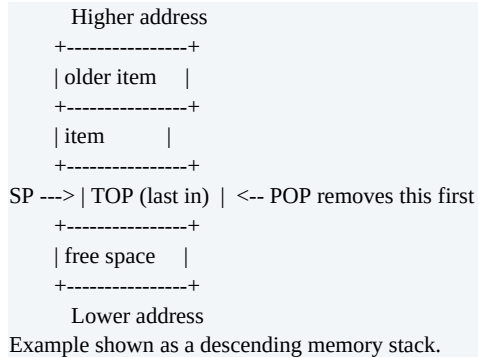
A control word is a binary word whose fields specify a set of simultaneous micro-operations in the CPU datapath. In a general register organization it commonly contains fields such as SELA and SELB for source registers, SELD for destination, OPR for ALU function, and SH for shift operation.

To determine a control word, first write the desired register-transfer operation. Identify the two required sources, select the destination, choose the ALU operation, and choose any shift. Encode each selection using the assigned field codes and concatenate the fields in the defined order. For example, $R1 \leftarrow R2 + R3$ requires SELA=R2, SELB=R3, SELD=R1, OPR=ADD and SH=NO SHIFT.

3. Define stack. Explain the stack organization.

A stack is a Last-In-First-Out (LIFO) storage structure. Data is inserted by PUSH and removed by POP. The Stack Pointer (SP) identifies the current top of stack. In a register stack, a set of CPU registers acts as stack storage and the pointer selects a register. In a memory stack, a region of main memory stores stack words and SP contains a memory address.

Depending on the architecture, a push may decrement SP then store data (descending stack) or store then increment SP (ascending stack). Stacks are used for subroutine return addresses, saved registers, local variables, parameters and interrupt context.



Memory stack concept

4. Explain the different instruction formats with examples.

Instruction format defines how bits of an instruction are divided into fields such as opcode, register numbers, addressing-mode bits and address/immediate data. Common formats are classified by the number of explicit address operands.

Format	General form	Example / meaning
Three-address	OP D,S1,S2	ADD R1,R2,R3 => R1=R2+R3
Two-address	OP D,S	ADD R1,R2 => R1=R1+R2
One-address	OP X	ADD X => AC=AC+M[X] in accumulator machine
Zero-address	OP	ADD => pop two stack items, push sum
Register/immediate	OP R,#data	MOV R1,#5

5. Explain the different types of instruction addressing modes.

Mode	Effective operand location	Example concept
Immediate	Constant is inside instruction	MOV R1,#25
Direct/absolute	Address field is effective address	LOAD R1,[1000]
Indirect	Address field points to a pointer	LOAD R1,[[1000]]
Register	Operand is in register	ADD R1,R2
Register indirect	Register contains memory address	LOAD R1,[R2]
Indexed	EA = address + index register	LOAD R1,ARRAY[R3]
Base/displacement	EA = base register + displacement	LOAD R1,8(R2)
Relative	EA = PC + signed displacement	conditional branch

Mode	Effective operand location	Example concept
Auto increment/decrement	Register indirect plus automatic pointer update	(R2)+ or -(R2)
Implied/stack	Operand is implied by instruction or stack top	CMA, PUSH/POP style

6. Explain the different Data Transfer and Manipulation instructions.

Data-transfer instructions copy information between registers, memory, stack and I/O without conceptually changing the bit pattern. Examples include MOV/LOAD/STORE, exchange, push, pop, input and output. Data-manipulation instructions transform data and are usually divided into arithmetic, logical and shift operations.

- Arithmetic: ADD, ADC, SUB, SBB, INC, DEC, multiply, divide, negate.
- Logical: AND, OR, XOR, NOT/complement, compare, test, bit set/clear.
- Shift/rotate: logical shift left/right, arithmetic shift right, rotate left/right, rotate through carry.
- Data transfer: register-register, memory-register, immediate load, stack transfer and I/O transfer.

7. What are control instructions? Explain the different types of program control instructions with their roles.

Program-control instructions alter the normal sequential flow of instruction execution or control processor state.

- Unconditional branch/jump: always loads a new PC value.
- Conditional branch: branches only when a flag or condition is satisfied.
- Call: transfers control to a subroutine while saving a return address.
- Return: restores the saved return address and continues the caller.
- Skip/loop instructions: conditionally skip or repeat based on a counter/condition.
- Interrupt/trap instructions: deliberately enter a service routine or software exception.
- Processor control: enable/disable interrupts, halt, no-operation, or change status/control state.

8. Define program interrupt. Explain the types of interrupt.

A program interrupt is an event that temporarily suspends normal program execution, transfers control to an interrupt service routine (ISR), and later returns to the interrupted program after saving/restoring the necessary state. Interrupts allow the CPU to respond efficiently to external devices, exceptional conditions and software requests.

- Hardware/external interrupt: generated by I/O devices, timers or external circuitry.
- Software interrupt: generated by an instruction in the program, often for system services.
- Internal interrupt/exception: generated by CPU-detected conditions such as divide-by-zero or invalid operation.
- Maskable interrupt: can be disabled or masked by software.
- Non-maskable interrupt: cannot normally be disabled and is reserved for urgent events.
- Vectored interrupt: ISR address is predefined or supplied by a vector; non-vectored interrupt requires another method to determine the ISR address.

9. Write short notes on: Status bit conditions, Conditional branch instructions, Subroutine Call and Return.

Status bit conditions: ALU operations produce flags such as Zero, Carry, Sign/Negative, Overflow and Parity. These bits summarize properties of the latest result and are used by later instructions.

Conditional branch instructions: A conditional branch tests one or more status conditions and changes the PC only if the condition is true; otherwise execution continues sequentially. Examples include branch-if-zero, branch-if-carry and branch-if-negative.

Subroutine Call and Return: CALL saves the address of the instruction following the call, commonly on the stack, and loads the subroutine entry address into PC. RETURN retrieves the saved address and resumes the caller. Nested procedures work naturally when return addresses are stacked in LIFO order.

10. Differentiate between CISC and RISC architecture.

Feature	CISC	RISC
Instruction set	Large and feature-rich	Smaller and simpler
Instruction length	Often variable	Often fixed or regular
Addressing modes	Many	Fewer
Memory operations	Instructions may operate directly on memory	Usually load/store architecture
Cycles per instruction	May vary widely	Many instructions designed for few cycles
Control	Often historically microprogrammed	Often hardwired/regular
Pipelining	Harder with irregular instructions	Easier due to regular formats
Code size	Often compact	May require more instructions
Examples	x86 family	ARM (classic RISC roots), RISC-V, MIPS

Unit Seven: Computer Arithmetic

1. Explain addition and subtraction of signed magnitude numbers with example.

In signed-magnitude representation, the most significant bit is the sign (0 for positive, 1 for negative) and the remaining bits store magnitude. For addition, first inspect the signs. If both signs are equal, add magnitudes and keep the common sign. If signs differ, subtract the smaller magnitude from the larger and use the sign of the operand having the larger magnitude.

Subtraction A-B is handled by changing the sign of B and applying the same signed-magnitude addition rules.

- Example: $(+5) + (+3)$. Magnitudes $0101 + 0011 = 1000$, sign 0 $\Rightarrow +8$.
- Example: $(+7) + (-3)$. Signs differ; $7 - 3 = 4$ and larger magnitude is positive $\Rightarrow +4$.
- Example: $(-7) + (+3)$. Signs differ; $7 - 3 = 4$ and larger magnitude is negative $\Rightarrow -4$.
- Example: $(+5) - (+8) = (+5) + (-8)$: $8 - 5 = 3$ with negative sign $\Rightarrow -3$.

2. Discuss the rules for handling the sign bit in each case.

- Addition with same signs: add magnitudes; result sign equals the common sign. Detect magnitude overflow separately.
- Addition with different signs: subtract smaller magnitude from larger magnitude; result sign equals the sign of the larger magnitude. If magnitudes are equal, result is zero.
- Subtraction A-B: complement only the sign of B, then perform signed-magnitude addition.
- Positive zero and negative zero can both exist in signed-magnitude representation; systems normally normalize the result to positive zero.
- The sign bit is never included as an ordinary magnitude bit during magnitude addition/subtraction.

3. Explain addition and subtraction using signed 2's complement representation. Show example for (+7)+(-3).

In two's-complement arithmetic, the same binary adder is used for signed addition and subtraction. To add two signed numbers, add all bits including the sign bit and discard any carry out of the most significant bit. Overflow occurs when two operands with the same sign produce a result with the opposite sign. To compute $A-B$, add A to the two's complement of B .

For 4-bit $(+7)+(-3)$: $+7 = 0111$, $+3 = 0011$, so -3 is two's complement 1101 . Then $0111 + 1101 = 1\ 0100$. Discard the carry out, leaving $0100 = +4$. The operands have different signs, so signed overflow cannot occur.

```

+7 = 0111
-3 = 1101
----
 0111
+ 1101
----
 1 0100
^^^ = 0100 = +4 (carry out discarded)

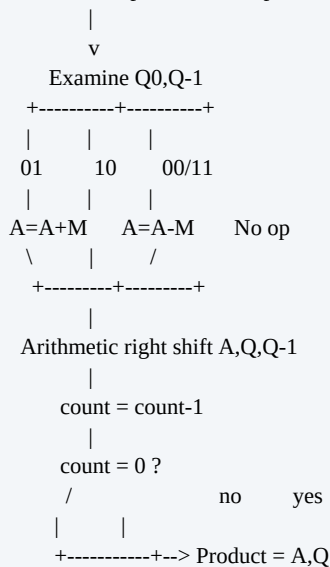
```

Two's-complement addition example

4. Describe Booth's Multiplication algorithm in detail. Present the flowchart and apply it to compute $(-5) \times (+3)$ using 4-bit 2's complement.

Booth's algorithm multiplies signed two's-complement numbers efficiently by examining adjacent multiplier bits. Use A as an accumulator, Q as the multiplier, M as the multiplicand, $Q-1$ as an extra bit, and a count n . Initially $A=0$, $Q-1=0$ and $\text{count}=n$. At each step inspect $Q_0, Q-1$: 01 means $A=A+M$; 10 means $A=A-M$; 00 or 11 means no arithmetic. Then perform an arithmetic right shift of the combined $A, Q, Q-1$ and decrement count. After n steps, the $2n$ -bit product is A, Q .

Start: $A=0$, $Q=\text{multiplier}$, $M=\text{multiplicand}$, $Q-1=0$, $\text{count}=n$



Booth multiplication flow

For $(-5) \times (+3)$ with 4-bit two's-complement: $M = -5 = 1011$, $Q = +3 = 0011$, $A = 0000$, $Q-1 = 0$.

Step	Q0 Q-1	Operation before shift	A after operation	After arithmetic right shift: A Q Q-1
Initial	-	-	0000	0000 0011 0
1	10	$A = A - M = 0000 -$	0101	0010 1001 1

Step	Q0 Q-1	Operation before shift	A after operation	After arithmetic right shift: A Q Q-1
		1011		
2	11	No operation	0010	0001 0100 1
3	01	A = A + M = 0001 + 1011	1100	1110 0010 0
4	00	No operation	1110	1111 0001 0

Final product A,Q = 1111 0001. As an 8-bit two's-complement number this equals -15, so $(-5) \times (+3) = -15$.

5. Explain the restoring division algorithm and perform the division of 11 by 3 using restoring method.

Restoring division uses registers A (partial remainder), Q (dividend/quotient), M (divisor), and a count. Initially A=0 and Q contains the dividend. For each bit: shift the combined A,Q left; subtract M from A; if A becomes negative, set the new quotient bit Q0=0 and restore A by adding M; if A is non-negative, set Q0=1. After n iterations, Q contains the quotient and A contains the remainder.

Initialize A=0, Q=dividend, M=divisor, count=n

```

  |
  v
Shift left (A,Q)
  |
  A = A - M
  |
  Is A negative?
  /           yes      no
  |           |
  Q0=0; A=A+M   Q0=1
  \           /
  +-----+
  |
  count=count-1
  |
  repeat n times
  |
  Quotient=Q, Remainder=A

```

Restoring division flow

For 11 / 3 using 4 bits: dividend Q=1011, divisor M=0011, A=0000.

Cycle	After left shift A,Q	A-M	Decision / restored A	Q after Q0 set
1	0001 0110	1110 (negative)	Restore A=0001; Q0=0	0110
2	0010 1100	1111 (negative)	Restore A=0010; Q0=0	1100
3	0101 1000	0010 (non-negative)	Keep A=0010; Q0=1	1001
4	0101 0010	0010 (non-negative)	Keep A=0010; Q0=1	0011

Final Q = 0011 = 3 and A = 0010 = 2. Therefore, 11 / 3 gives quotient 3 and remainder 2.

Unit Eight: Input/Output Organization and Memory Organization

1. Explain the concept of interrupt-driven I/O.

Interrupt-driven I/O allows the CPU to start or enable an I/O operation and continue executing other instructions instead of repeatedly polling the device. When the device becomes ready or completes the operation, it asserts an interrupt request. The CPU completes the current instruction, saves enough processor state, acknowledges the interrupt, and branches to an interrupt service routine. The ISR transfers data or handles the event, clears the interrupt condition, restores the saved state, and returns.

Compared with programmed polling, interrupt-driven I/O uses CPU time more efficiently when device events are relatively infrequent. However, each interrupt has service overhead and very high interrupt rates can reduce performance.

2. Describe peripheral devices, I/O bus, I/O interface modules, and differentiate Isolated I/O and Memory-Mapped I/O.

Peripheral devices are external or auxiliary devices that provide input, output, storage or communication, such as keyboards, displays, disks, printers, sensors and network interfaces. An I/O bus carries data, address/selection and control information between the processor/system bus and I/O interfaces. An I/O interface module adapts device-specific timing and electrical behavior to the CPU bus and normally contains data registers, status registers, control registers, address-decoding logic and handshaking circuitry.

Feature	Isolated (Port-Mapped) I/O	Memory-Mapped I/O
Address space	Separate I/O address space	Shares main memory address space
Instructions	Special IN/OUT instructions	Normal load/store instructions
Control signals	Separate I/O read/write distinction	Memory read/write mechanism
Memory address space	Not consumed by ports	Some addresses assigned to devices
Programming flexibility	Fewer addressing modes in many CPUs	All memory addressing modes can often be used

3. Explain Direct Memory Access (DMA).

Direct Memory Access is a technique in which a DMA controller transfers blocks of data directly between an I/O device and main memory without requiring the CPU to execute an instruction for each byte or word. The CPU first programs the DMA controller with source/destination address, transfer count and direction. The controller then requests control of the system bus. When the CPU grants the bus, the DMA controller generates addresses and read/write control signals and performs transfers. On completion it releases the bus and may interrupt the CPU.

DMA is especially useful for high-speed block devices such as disks, audio interfaces and network adapters. Common transfer modes include burst/block mode, cycle stealing and transparent DMA.

4. Explain the concept of interrupt-driven I/O. (Repeated in the assignment sheet)

Same concept as Question 1: the device interrupts the CPU when service is required, so the CPU does not waste cycles continuously polling the device. The interrupt service routine performs the required I/O work and then returns to the interrupted program.

5. Differentiate between maskable and non-maskable interrupts.

Point	Maskable Interrupt	Non-Maskable Interrupt
Software disabling	Can be disabled/masked	Normally cannot be disabled by

Point	Maskable Interrupt	Non-Maskable Interrupt
		ordinary interrupt mask
Priority/use	Normal peripheral events	Critical/high-priority events
Typical examples	Timer, serial port, ordinary I/O requests	Power failure, severe hardware fault
Response guarantee	May be delayed while masked	Designed for urgent response
8085 example	RST 7.5, RST 6.5, RST 5.5, INTR	TRAP

6. What are the different types of interrupts, and how are they used in I/O operations?

Interrupts can be classified as hardware versus software, internal versus external, maskable versus non-maskable, and vectored versus non-vectored. In I/O systems, hardware interrupts are the most common: a device controller requests attention when input data is available, an output buffer becomes empty, a transfer completes, or an error occurs. A vectored interrupt lets the processor reach the correct ISR efficiently. Masking allows software to temporarily block selected device interrupts while a critical section is executing. Priority logic resolves simultaneous requests from multiple devices.

7. What are DMA controllers, and how do they facilitate faster data transfer?

A DMA controller is dedicated hardware that temporarily becomes bus master and moves data between memory and an I/O device. Its registers usually include memory address, byte/word count, transfer mode and status. By generating the bus addresses and read/write strobes in hardware, it avoids the repeated fetch-decode-execute overhead of CPU-controlled transfers. The CPU is involved mainly during setup and completion notification. This greatly improves throughput for large blocks and frees the processor to perform other work, although DMA bus use can temporarily delay CPU memory accesses.

8. What are the different types of interrupts, and how are they used in I/O operations? (Repeated in the sheet)

This repeats Question 6. For I/O, interrupts provide an asynchronous notification mechanism. Peripheral controllers raise requests, priority/mask logic decides which request is accepted, the CPU runs the corresponding ISR, and the ISR exchanges data/status with the device before returning.

Unit Nine: Pipelining

1. What is parallel processing? Explain the benefits of parallel processing.

Parallel processing is the simultaneous or overlapping execution of multiple operations by using more than one processing element, functional unit, core, machine, or pipeline stage. Work may be divided at the bit, instruction, data, task, thread or process level. The goal is to exploit independent operations so more computation is completed per unit time.

- Higher throughput and reduced execution time for parallelizable workloads.
- Ability to solve larger and more complex scientific, graphics, AI and data-processing problems.
- Better utilization of multiple CPU cores and functional units.
- Improved responsiveness by running independent tasks concurrently.
- Scalability by adding processors/cores, subject to communication and serial-work limits.
- Potential reliability improvements when redundant processors or tasks are used.

2. Explain the classifications of parallel processing by M. J. Flynn.

Flynn's taxonomy classifies computer organizations according to the number of concurrent instruction streams and data streams.

Class	Instruction streams	Data streams	Meaning / example
SISD	Single	Single	Conventional sequential uniprocessor
SIMD	Single	Multiple	One instruction operates on many data items; vector units/GPUs
MISD	Multiple	Single	Multiple operations on one data stream; rare, sometimes fault-tolerant/specialized pipelines
MIMD	Multiple	Multiple	Multiple processors execute different instructions on different data; multicore/SMP/clusters

3. What is pipelining? Explain the role of pipelining in computing.

Pipelining is a form of temporal parallelism in which a task is divided into stages and different stages work on different items at the same time. In an instruction pipeline, while one instruction is being executed, another may be decoded and another fetched. After the pipeline fills, ideally one result can complete every clock cycle even though a single instruction still passes through all stages.

Pipelining raises throughput, improves utilization of hardware stages and permits higher clock rates when a long combinational path is divided by pipeline registers. Real speedup is limited by unequal stage delays, pipeline-register overhead, resource conflicts, data dependencies and branches.

4. Define instruction pipeline. Explain the four-segment instruction pipeline.

An instruction pipeline divides instruction processing into sequential stages so several instructions can be in different stages concurrently. A common four-segment organization is FI (Fetch Instruction), DA (Decode instruction and calculate effective Address), FO (Fetch Operand), and EX (Execute instruction). Pipeline registers separate stages. On each clock, an instruction advances one stage when no hazard forces a stall.

```

Clock cycle:  1   2   3   4   5   6   7
Instruction I1: FI --> DA --> FO --> EX
Instruction I2:      FI --> DA --> FO --> EX
Instruction I3:          FI --> DA --> FO --> EX
Instruction I4:              FI --> DA --> FO --> EX

```

FI = Fetch Instruction
 DA = Decode / effective Address
 FO = Fetch Operand
 EX = Execute

Four-segment instruction pipeline

5. Explain the different pipeline hazards (conflicts).

A pipeline hazard is a condition that prevents the next instruction from executing in its intended clock cycle.

- Structural/resource hazard: two stages need the same hardware resource simultaneously, such as one memory for both instruction fetch and data access.
- Data hazard: an instruction depends on a value produced by an earlier instruction. RAW (read after write) is the most common true dependency; WAR and WAW can occur in more advanced pipelines.

- Control/branch hazard: the correct next instruction address is unknown because of branch, jump, call, return or exception.
- Hazards are handled using stalls/bubbles, forwarding/bypassing, register renaming, duplicated resources, scheduling, branch prediction, delayed branches or speculative execution.

6. How to handle the branch instruction in pipeline? Explain.

A branch creates a control hazard because the processor may have already fetched instructions from the sequential path before the branch decision and target are known. Several techniques are used to reduce the penalty.

- Stall until branch resolution: simplest but wastes cycles.
- Flush on taken branch: fetch sequentially; if branch is taken, discard wrong-path instructions and restart at target.
- Resolve branch earlier: move comparison/target calculation to an earlier stage to reduce lost cycles.
- Static prediction: always predict taken/not-taken or use compiler information.
- Dynamic branch prediction: hardware predicts using recent branch history and a branch target buffer.
- Delayed branching: compiler fills one or more branch-delay slots with useful independent instructions.
- Speculative execution: execute predicted-path instructions before certainty, then commit them only if prediction was correct.

End of Assignment

All distinct questions from Units 1-9 and all listed 8085 laboratory tasks have been answered.